

# Congressional Bill Status Prediction Using CNNs and Bayesian Modeling

Jeremiah Payeur & Madelyn Redick

DS4420: Machine Learning 2 with Dr. Gerber

December 4, 2025

## Introduction And Literature Review

The United States Congress proposes thousands of bills every year that are the basis for the legal system of the country. However, less than 10% of bills on average end up passing into law. The objective of this paper is to take the text that makes up these bills and use the machine learning methods of Bayesian Modeling and Convolutional Neural Networks (CNNs) to predict whether a bill will become law. Lawmakers could use these models to pass through bills before proposing them to see what they will most likely become before introducing them, giving them a better chance at having them passed while also not wasting as much time on bills that will likely not pass. The aim is to optimize not just for overall accuracy but also for accuracy of the predictions of “Became Law” status. This is because if a lawmaker can be confident that when they see the outcome “Became Law,” it will be accurate, it will make the models more useful. The Bayesian Model, while for general purpose classification would not be entirely useful, does a good job optimizing for the “Became Law” category. The multi-class classification CNN has high accuracies for predicting most classes including “Became Law”, and the binary classification CNN has an even higher accuracy for predicting the law class.

The Bayesian Text Classification Model uses a similar methodology to the one discussed in Rudner’s *Automated Essay Scoring Using Bayes’ Theorem*. Rudner’s methodology uses a Multivariate Bernoulli to set up the likelihood function of the text. The Multivariate Bernoulli relies on finding the optimal number of words to take as an input, each word with a Bernoulli, 0 or 1, output that helps find the overall probability of the likelihood. While Rudner’s optimal number of words was found to be about 200 for the classification of essays, this paper finds that only 20 words is much more effective as an input. Rudner also demonstrates that it is important to find the optimal number of words when looking at stemmed, or root words, versus unstemmed words. Using stemmed words and treating all root words as the same, this means that if there is a very frequently used word with many unstemmed variants, it reduces the times this root is used in the input from many to one. This reduction in input data is beneficial when attempting to not overfit to a specific word, but can also cause lost insights if the use of the word in unstemmed contexts varies heavily. While Rudner found that using unstemmed words worked best, the Bayesian Model for this paper uses stemmed words. Further, at the core of Bayesian modeling, it is necessary to set up a prior belief. In Rudner’s methodology, he suggests updating the prior with each iterative posterior and putting equal weight on the prior and the data for each

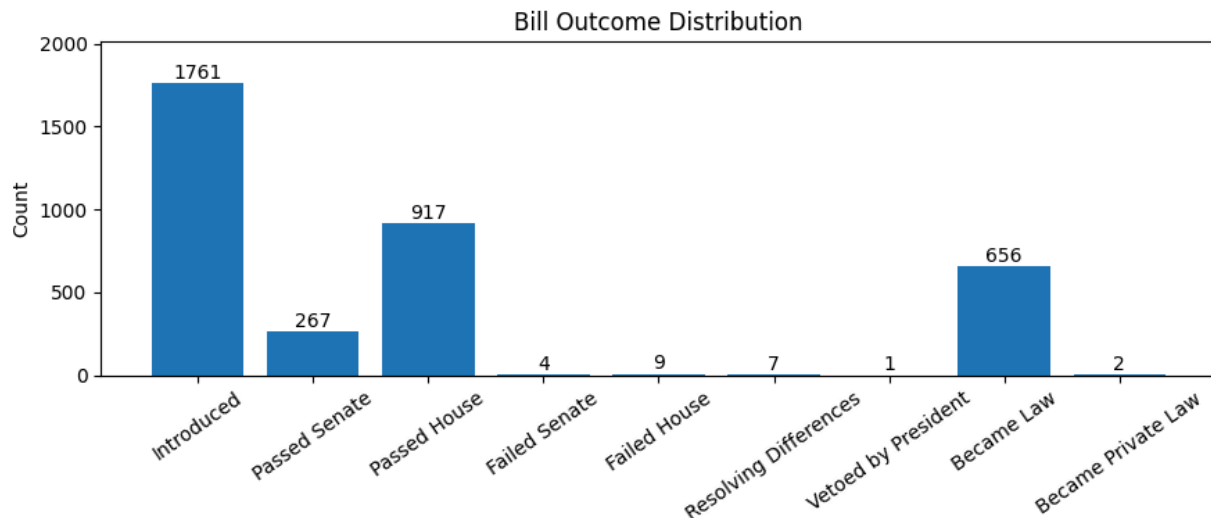
prediction. However, through testing, what may have been appropriate for essay classification is seen to not be effective for bill classification because of how much the data shifts the posterior, using previous posteriors as priors yielded poor results (Rudner and Liang 2002, 6-11).

Another method that could be used to predict the passage of congressional bills is a Convolutional Neural Network because they are useful for extracting meaningful features from texts, including long and complex legislative documents. The work done by Korn and Newman to classify the status of congressional roll call votes involved building a hybrid CNN-LSTM (Long-Short Term Memory) model that recognizes features in sparse word vector spaces and identifies abstract patterns within them. They used several techniques in the model's architecture including 1-dimensional convolution layers with ReLU activation functions, Max Pooling layers, and initializers that adapt weights to the input data's shape (Variance Scaling Kernel). Additionally, they used L1 and L2 regularization (Elastic Net), a sigmoid output function, and 10-fold cross-validation. The model resulted with an accuracy of 67.32% for classifying whether or not a bill was passed. The authors of "A Deep Learning Model to Predict Congressional Roll Call Votes from Legislative Texts" claim that the bills alone hold enough information for their model to recognize sophisticated features from the data representations transformed into word vector spaces (Korn, Newman 2020). Certain aspects of this model structure, such as the activation and output functions and convolutional design, influenced the architecture of the CNNs.

## **Methodology and Data Collection**

Congress provides an API to access congressional and bill related data on their website, however, because this API only allows for 250 requests at a time, it was much more effective to download the data once and store it locally. This paper tests Bayesian Text Classification on the entire bill text while the CNN relies on the summaries of the bills. To collect the entire bill text, first information about the bill such as the congress it was introduced in, whether it was introduced to the House or Senate, and how far it has made it in its legislative process, are downloaded. Then, with this identifying information, PyPDF2 is used to read in the text from formatted URLs. Additionally for the Bayesian Model, text was cleaned of excess punctuation and stopwords and stemmed when needed. The summaries for the CNN are downloaded directly from the congressional website and cleaned of absent summaries and HTML tags still present in

the text. The low proportion of bills that become laws is reflected in the class imbalance of the datasets, demonstrated in the bar plot from the dataset used on the Bayesian Model below.



The Bayes Text Classification Model first requires formatting the input data, the text, into a form that can be used for calculations. First stop words such as “a”, “the,” and “bill” are removed, then each unique word,  $w$  is treated as a Bernoulli variable with a 1 if it appears in a bill and 0 if it does not. The most frequent  $n$  words are used for inputs and are what the conditional likelihoods are based off of. The conditional likelihood for each word, the probability of the word occurring given the outcome are then calculated based on the training data, call this  $p(w|o)$ . The likelihood for the Bayesian model is then  $p(t|o_j) = \prod_{i=1}^n p(w_i|o_j)(a_i) + (1 - p(w_i|o_j))(1 - a_i)$ , the same as the product of the individual Bernoulli likelihoods, where  $a_i$  is if the word appeared in the document that is being classified and  $o_j$  is the  $j$ th outcome that the likelihood is being calculated for. This likelihood function  $p(t|o_j)$  is then multiplied by the prior  $p(o_j)$  and the sum of all of these joint distributions of  $o$  are then summed together to find  $p(t)$ . These probabilities come together to calculate the posterior  $p(o|t) = p(t|o)p(o)/p(t)$ , the outcome with the highest posterior probability is then used for prediction.

Once the Bayesian model is set up mathematically, it is important to consider how changing hyperparameters might impact the final result. The Bayesian Text Classifier used 4,000 samples of bills and trained the conditional likelihood of the words on 3,200 of them (80%). An additional 400 (10%) of the samples were then used to tune the hyperparameters, such as number of words, prior, and stemmed or unstemmed. Each stemmed and unstemmed result were plotted out over different word vector lengths in groups of priors. The prior construction took a bit more

thought. One of the aspects to optimize for is the probability of predicting “Became Law” is correct which meant that the prior on “Became Law” likelihood could not be too high because that would cause the model to over-predict “Became Law.” However, another aspect to optimize for is the overall percent of bills that actually “Became Law” that were properly classified. This condition meant that the prior on “Became Law” still needs to be sufficiently high that we properly classify bills in this category. The balance of ensuring that the predicted was high while also ensuring the accuracy of “Became Law” bills was high meant that many priors were selected from and tested rather than starting with a uniform prior as Rudner did. After stemmed vs. unstemmed, the optimal prior, and the optimal n were selected, the Bayesian model was tested on the last 400 samples, samples that had never been seen by the model before. One important consideration is that some of the bills do not yet have their full text available as PDFs, so the actual predicted samples in the test set are slightly smaller.

The intention behind using a CNN for bill text classification is to identify specific aspects of the text that influence the outcome. Two different CNNs were designed, one a multi-class text classifier and one a binary text classifier; both have the same architecture but different output and activation functions. Each bill summary text was cleaned of HTML tags and vectorized, that is, turned into a vector where each element is a word from the text. These vectors were set to be a specific length (200) to maintain dimensions during model construction and training. If the text was shorter than 200 words, it was padded with empty elements, and if it was longer it was truncated. The word vectors are then converted to hold integer IDs representing unique words. The input data for both models was a matrix where each row is a bill’s vector of word IDs.

Each model architecture begins with an embedding layer that feeds into three one-dimensional convolutional layers. The embedding layer converts word IDs to semantic meanings by adding a depth layer (of size 100). Semantic meanings are learned through backpropagation of the model; words that appear in similar contexts are predictive of the same output classes and will have closer embeddings. This is an important step because it allows the CNNs to learn relationships between words. The convolutional layers are parallel rather than linearly chained because their goal is to find different types of patterns within the same input. They have kernel sizes of 2, 3, and 5, which capture 2-gram, 3-gram, and 5-gram phrases that likely contribute to the bill’s outcome. A ReLU activation function is then applied to the convolutional output and fed into a Global Max Pooling layer that reduces each weight matrix

output to the strongest activation produced. Since each pooling layer generates 100 features, concatenating them results in a combined feature vector of 300 n-grams (100 2-grams, 100 3-grams, and 100 5-grams). A dropout regularization layer is applied to this vector, randomly setting 30% of the weights to 0 to prevent overfitting and improve generalization of the model. A linear function is applied to the fully connected layer to map the 300 dimension feature vector to a single score per class. These outputs are referred to as logits, scores of how strongly the model thinks each class is correct.

The multi-class CNN uses softmax as an output function and the cross-entropy loss function. Programmatically, R's cross-entropy loss function internally computes softmax values from logits, which is why the model outputs them rather than directly using an output function. The binary classification CNN uses the sigmoid output function and binary cross-entropy loss function. This loss function programmatically has the same internal application of the output function, requiring logits as an input to binary cross-entropy. The significant class imbalances in the dataset were accounted for in the loss functions. Each class was assigned a weight inversely proportional to its frequency in the data, which is passed to the loss functions which multiplies the loss for each sample by the weight corresponding to its true class. Both models use Adam (adaptive movement estimation) as their optimization function, which is a variant of stochastic gradient descent that adapts the learning rate for each parameter. This optimizer is especially useful in CNN text classification cases due to the high number of parameters.

Both models were trained over 75 epochs with 70% of the dataset, while a test and validation set were each created with 15% of the data. The model was evaluated on the validation set within the training loop to detect overfitting early and make sure that improvements in the training loss reflected true improvements in performance on unseen data. A batch size of 32 was used during training to reduce computational load and escape local minima during gradient descent.

## **Results and Interpretation**

The Bayesian Model tested multiple priors, n's (number of words), and stemming words, using validation data, optimizing for overall accuracy, "Became Law" accuracy, and "Became Law" predictive accuracy [Figure 1]. When looking at strictly predictive accuracy, stemmed does consistently better than unstemmed, across n and across priors. Stemmed accuracy peaks at

52-58% depending on the prior, but has peaks that are consistently at  $n=5$  and will drop off as  $n$  increases. This indicates that more information does not necessarily help the model with greater predictive accuracy, and that as inputs increase, it is more likely to confuse the model than to enhance it. The fact that stemmed words have higher accuracy also supports this fact because the unstemmed words can create clutter that reduces accuracy. With that, it is also important to note that the steepest drop off occurs as the first few words are added, and that overall accuracy only decreases by  $\sim 10\%$  from  $n=5$  to  $n=100$ . This means that when optimizing other outputs, it may be worth it to sacrifice a small portion of overall accuracy to increase the “Became Law” overall or predicted accuracy.

The “Became Law” overall accuracy followed more of a parabolic trend and was heavily influenced by the prior. For instance, on stemmed data the prior that gave “Became Law” a 98% chance to start, weighted equally with the sample, found “Became Law” accuracy from 30-55%, while the prior that placed a 50% chance to start had an accuracy from 22-37%, and the uniform prior had accuracies from 22-32%. This shows what is expected in that, as when there is imperfect classification of a specific class, increasing the prior on this class will also increase how many within that class are properly classified. However, this has an inverse relationship with “Became Law” predictive accuracy, which as the prior on “Became Law” increases, and the number of “Became Law” predictions with it, the percent of those predictions that are right will decrease. This is seen in the stemmed data where when the prior is placed at 98%, only 75-90% of the predictions are accurate, whereas when the prior is placed at 50% between 90-100% of predictions of “Became Law” are accurate. This means that when “Became Law” is predicted in a 98% prior, the bill is much less likely to have become law than if the bill is predicted to have “Became Law” with a uniform prior.

Observing these results, the optimal prior on “Became Law” is at 50%, with stemmed words, and an  $n$  of 20. Using this model, the test set accuracy recorded 57.85%, the “Became Law” overall accuracy is 44.44%, and the “Became Law” predicted accuracy is 96.55%. This means that the overall coverage on bills that became law is 44.44% (4/9 bills that do become law are predicted as so), and that when the model predicts something to become law it does 96.55% of the time. This indicated to lawmakers that if they were to use this model on a new bill and it classifies the bill as “Became Law” then it has a 96.55% chance to become law. Additionally, if the bill will eventually become law, there is a reasonable 4/9 chance that the model classifies it as

such. However, the Bayesian Model uses past data so future predictions may not be as accurate. Another consideration is that when using these optimized parameters, the model only predicts “Became Law” and “Introduced.” This means that binary classification could have been used to yield the same results, and the categorical aspects are being “lost” in that they are not predicted. When testing other priors, ns, and stemmed vs. unstemmed, other categories were predicted as well, meaning that with enough fine tuning there exists a better Bayesian that classifies beyond these two categories. However, when optimized it ends up not predicting other categories for the test set despite their existence.

The overall accuracy for the multi-class text classification CNN was 79.89%, correctly identifying bills that became law in 74.43% of cases. This model’s confusion matrix resulting from evaluation on the test set (736 samples) demonstrates that the model correctly predicts most outcomes very well, especially for “Passed Senate” and “Passed House” [Figure 2]. It most commonly inaccurately predicts bills that become laws as “Passed House”, which makes sense because bills that become laws have to be approved by both chambers to become a law, so language that is consistent with bill passage would also likely be found in proposals approved by a chamber. The binary text classification CNN achieved an overall accuracy of 89.81%. It correctly identified bills that became law in 93.65% of cases, indicating that the model successfully captures key textual patterns associated with successful legislation. Initially, the CNNs did not validate during training, which resulted in overfitting. The multi-class CNN had an overall accuracy of 93.3% and an accuracy for predicting “Become Law” of 90.7%. The binary CNN had an overall accuracy of 97.3% and an accuracy for predicting if a bill became a law of 95.9%. While these accuracies alone seem to indicate the models are very useful, they are deceptive because the models would not be able to generalize well to new bills.

The multi-class CNN’s high performance on “Passed Senate” and “Passed House” outcomes suggest that the model learned to recognize consistent structural language that exists in bill summaries that successfully moved through each chamber. This model’s common error was predicting “Passed Senate” and “Passed House” as “Became Law”, which is meaningful because it shows that the textual features associated with bill passage overlap with those associated with intermediate stages of passage. The poorer performance on uncommon classes, such as “Failed House” and “Resolving Differences” (both with a 16.67% accuracy) are due to the outcome imbalances since there is not enough data from these classes for the model to learn

distinguishable language from. The binary classification model performs better than the multi-class classification model because it has a simpler decision boundary; it only has to decide if a bill belongs to one of two classes, while the multi-class model must decide among nine classes. This improvement implies that the textual features that distinguish successful laws from failed bills are strong when treated as a binary signal. The CNNs capture n-gram patterns that reflect a real legislative document rather than simply memorizing words.

## **Conclusions and Future Work**

Both the Bayesian Model and CNN can provide valuable insights to lawmakers creating legislation, having high predictive “Became Law” accuracy, however there are improvements that can still be made. The Bayesian Model relies on assumptions such as independence in words, which, while stemming helps with, there is still likely not full independence within the input vector. Improvements to this can be made by seeing which input words might be highly dependent on each other, and removing one of the words if they meet a certain threshold. This would help enhance independence and the end probability predictions. Another assumption that the Bayesian model makes is within the prior, the prior was adjusted to account for the overprediction of certain categories in the data. This overprediction may have been caused by a lack of independence, so it is important to continue to test priors using validation data. Specifically, further work needs to be done to find the optimal prior that spreads out the current "Introduced" guesses into the other categories as well. This can be done by reducing the prior on "Introduced" while increasing other categories that were not predicted. This was partially tested with one of the 98% "Became Law" prior models, and helped boost the overall accuracy. However, because the other metrics regarding "Became Law" were not as strong, this prior was not chosen, but it provides a framework for a method to increase overall accuracy in the future.

Although overall accuracy for both CNNs decreased after cross-validation was introduced, these models are better for usage in real Congress due to their improved generalization abilities. An adjustment that could improve the CNNs further is adding a 4th parallel convolutional layer with a kernel size of 1 to capture specific words that influence the outcome. Currently, the models are only capable of capturing n-grams, combinations of multiple words, but it is possible that certain words contribute to the probability of a bill becoming a law. A convolutional layer with a larger kernel size, such as 8 or 10, was considered, but if there are

any important longer phrases, they will be captured by sequences of 3-grams or 5-grams. These models could also be extended to pull out the most significant n-grams that predict bill outcome. This would be especially helpful to Congress members while actively writing bills by providing them with a phrase bank to reference, increasing the odds that their bill makes it further in its legislative process.

Although the models are not directly comparable because they had different versions of input text, the better performance of the CNNs suggests that using bill summaries for predicting outcomes is more useful. However, in order for the models to be applied in Congress, bills must have a summary, which is not always the case. The Bayesian Model is effective in limiting type I error which allows lawmakers to have strong confidence in the model when it predicts “Became Law.” The overall predictive accuracy for the Bayesian Model, and type II error found in “Became Law” are future points of improvement. The multi-class text classification CNN is helpful in determining what outcome a bill has including “Became Law”, but the binary CNN would be recommended for practical use since the ultimate goal of lawmakers is to have their bills approved and the model performs better when considering a binary outcome.

## **Appendix**

*Figure 1:*

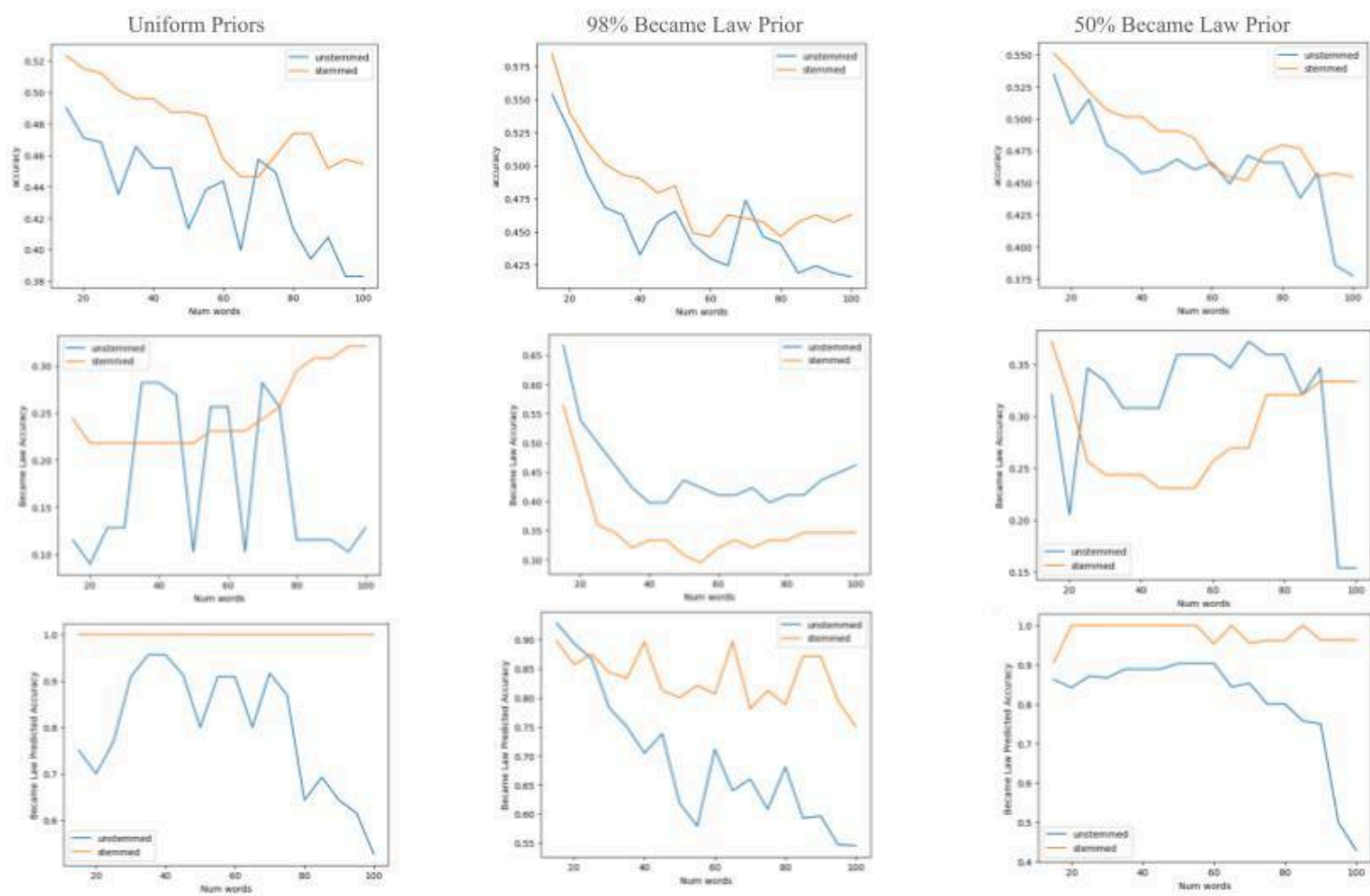


Figure 2:

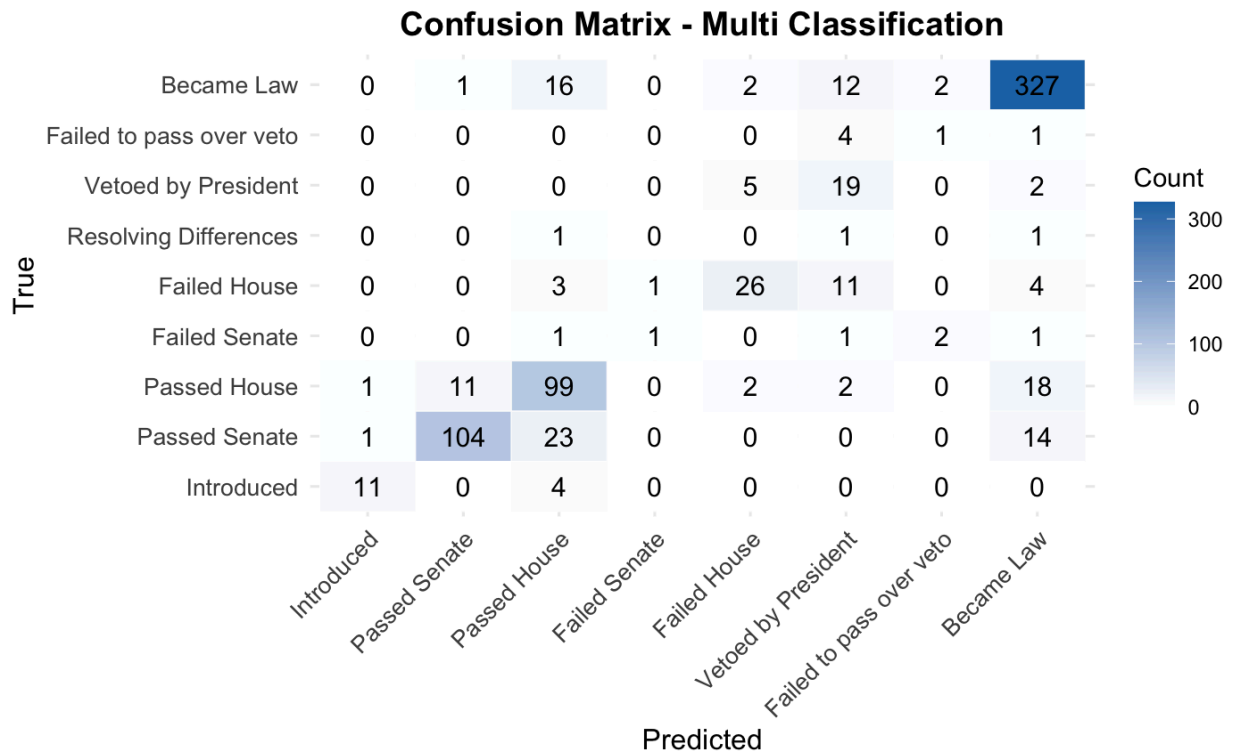


Figure 3:

